

1. A newly inserted heap element is initially placed at the _____ position/index of the array.

Solution: Last \ bottom-right \ (size+1)th

Explanation: A new element is added at the end to maintain the complete binary tree structure.

2. The worst-case space complexity of standard/original Heap Sort is $O(\text{_____})$ and modified Heap Sort is $O(\text{_____})$.

Solution: $O(1)$ and $O(n)$

Explanation: Modified Heap Sort creates a separate heap object H with an internal array of size $\backslash(n\backslash)$, while Standard Heap Sort sorts in place within the original array.

3. A heap is a complete binary tree that maintains the heap property. True / False

Solution: True

Explanation: A heap must satisfy both the complete binary tree structure and the heap property.

4. In a max heap, the left child must always be greater than the right child. True / False

Solution: False

Explanation: Both children must be less than or equal to their parent, but their values do not need to follow any order relative to each other.

5. In a min heap, the value of a parent node is:

A. Greater than or equal to its children B. Less than or equal to its children C. Always equal to its children

Solution: B. Less than or equal to its children

Explanation: In a min heap, every parent value is less than or equal to the values of its children.

6. Why is an array generally preferred over a linked list for implementing a heap?

A. Arrays allow direct parent and child access using indices B. Linked lists cannot store tree elements

Solution: A. Arrays allow direct parent and child access using indices

Explanation: The parent and child positions can be calculated directly from an element's array index.

7. Standard/original Heap Sort using a max heap sorts the array in: A. Ascending order B. Descending order C. Random order

Solution: A. Ascending order

Explanation: The maximum element is repeatedly moved to the end, producing an ascendingly sorted array.

8. A heap contains 31 nodes. What is its height? A. 4 B. 5 C. 31

Solution: A. 4

Explanation: A complete binary tree with 31 nodes has 5 levels and a height of 4 when height is counted by edges.

A heap contains 31 nodes. What is its height? A. 4 B. 5 C. 31

1 A heap is a **complete binary tree**.

2 Level-by-level node count in a complete binary tree:

Level	Nodes at that level	Cumulative total
Level 0 →	1 node	1
Level 1 →	2 nodes	1 + 2 = 3
Level 2 →	4 nodes	3 + 4 = 7
Level 3 →	8 nodes	7 + 8 = 15
Level 4 →	16 nodes	15 + 16 = 31

3 $1 + 2 + 4 + 8 + 16 = 31$ nodes → 4 This means there are **5 levels**.

5 Height = number of edges on the longest root-to-leaf path = **levels - 1**
So, Height = $5 - 1 = 4$

6 **Answer: A. 4**

A complete binary tree with 31 nodes has **5 levels** and a **height of 4** when height is counted by edges.

9. A max heap contains: [X, 50, 30, 40, 10, 20]. The value of the node at index 4 is increased from 10 to 45. Which operation should be used to restore the heap property? A. max-heapify B. heap-increase-key

Solution: B. heap-increase-key

Explanation: When a value is increased in a max heap, it may need to move upward toward the root.

A max heap contains: [X, 50, 30, 40, 10, 20]. The value of the node at index 4 is increased from 10 to 45. Which operation should be used to restore the heap property? **A. max-heapify**
B. heap-increase-key

1 Initial max-heap (1-based indexing).

Index	0	1	2	3	4	5
Value	X	50	30	40	10	20

2 Increase key at index 4.

Increase key: 10 → 45

Index	0	1	2	3	4	5
Value	X	50	30	40	45	20

3 Heap property is violated.

The node at index 4 has value **45**, which is greater than its parent **30**. Max-heap property is violated.

$45 > 30$ (Violation)

4 Correct operation to use:

Use heap-increase-key (heapify up / swim)

5 Move up (heapify up / swim).

1 Compare with parent: $45 > 30$ So, swap.

2 After swap:

6 Final max-heap.

Index	0	1	2	3	4	5
Value	X	50	45	40	30	20

7 **Answer: B. heap-increase-key**

When a value is increased in a max heap, it may need to move upward toward the root, so **heap-increase-key** is used.

10. Consider the max heap: [X, 50, 30, 40, 10, 20, 35]. After `extractMax()`, which value is moved to the root before applying `sink()`?
A. 10 B. 30 C. 35

Solution: C. 35

Explanation: During `extractMax()`, the last heap element is moved to the root before the `sink()` operation.

Consider the max heap: [X, 50, 30, 40, 10, 20, 35]. After `extractMax()`, which value is moved to the root before applying `sink()`? **A. 10 B. 30 C. 35**

1 Initial max heap

Array (1-based indexing)

Index	0	1	2	3	4	5	6
Value	X	50	30	40	10	20	35

Binary Tree

```

graph TD
    50((50)) --- 30((30))
    50 --- 40((40))
    30 --- 10((10))
    30 --- 20((20))
    40 --- 35((35))
        
```

2 Step 1: Remove the maximum value (50) from the root.

3 Step 2: Move the last heap element (35) to the root position before sinking.

This is the value moved to the root before sink().

4 Step 3: Temporary heap after moving 35 to the root (before sinking).

Array (1-based indexing)

Index	0	1	2	3	4	5	6
Value	X	35	30	40	10	20	

5 Step 4: sink() / heapify down.
Compare 35 with children 30 and 40 → swap with 40 (the larger child).

6 Final heap after `extractMax()` and `sink()`

Binary Tree

```

graph TD
    40((40)) --- 30((30))
    40 --- 35((35))
    30 --- 10((10))
    30 --- 20((20))
        
```

Array (1-based indexing)

Index	0	1	2	3	4	5	6
Value	X	40	30	35	10	20	

Answer: C. 35

During `extractMax()`, the last heap element (35) is moved to the root before the `sink()` operation.

```

public static int minCost(int[] ropes) {
    if (ropes == null || ropes.length <= 1) {
        return 0;
    }
}

```

```

MinHeap minHeap = new MinHeap();

```

```

// Insert all rope lengths into the MinHeap
for (int i = 0; i < ropes.length; i++) {
    minHeap.insert(ropes[i]);
}

```

```

int totalCost = 0;

```

// Either of the following loop conditions is correct:

// Option 1: // for (int i = 0; i < ropes.length - 1; i++)

// Option 2:

```

while (maxHeap.heapSize() > 1) {
    int firstMin = minHeap.extractMin();
    int secondMin = minHeap.extractMin();
}

```

```

int combinedLength = firstMin + secondMin;
totalCost += combinedLength;

```

```

minHeap.insert(combinedLength);
}

```

```

return totalCost;
}

```

```

public static int maxCost(int[] ropes) {
    if (ropes == null || ropes.length <= 1) {
        return 0;
    }
}

```

```

MaxHeap maxHeap = new MaxHeap();

```

```

// Insert all rope lengths into the MaxHeap
for (int i = 0; i < ropes.length; i++) {
    maxHeap.insert(ropes[i]);
}

```

```

int totalCost = 0;

```

// Either of the following loop conditions is correct:

// Option 1: // for (int i = 0; i < ropes.length - 1; i++)

// Option 2:

```

while (maxHeap.heapSize() > 1) {
    int firstMax = maxHeap.extractMax();
    int secondMax = maxHeap.extractMax();
}

```

```

int combinedLength = firstMax + secondMax;
totalCost += combinedLength;

```

```

maxHeap.insert(combinedLength);
}

```

```

return totalCost;
}

```